

Covert tree to BST

```
// Inorder traversal to store the nodes in a vector
void inorder(Node* root, vector<int>& nodes) {
    if (root == nullptr) {
        return;
    }
    inorder(root->left, nodes);
    nodes.push_back(root->data);
    inorder(root->right, nodes);
}

// Inorder traversal to convert tree
// to BST.
void constructBST(Node* root, vector<int> nodes, int& index) {
    if (root == nullptr) return;

    constructBST(root->left, nodes, index);

    // Update root value
    root->data = nodes[index++];

    constructBST(root->right, nodes, index);
}

// Function to convert a binary tree to a binary search tree
Node* binaryTreeToBST(Node* root) {
    vector<int> nodes;
    inorder(root, nodes);

    // sort the nodes
    sort(nodes.begin(), nodes.end());

    int index = 0;
    constructBST(root, nodes, index);
    return root;
}
```

```
// Function to print the inorder traversal of a binary tree
void printInorder(Node* root) {
    if (root == NULL) {
        return;
    }
    printInorder(root->left);
    cout << root->data << " ";
    printInorder(root->right);
}
```

```
Node* leftMost(Node* node) {  
    Node* curr = node;  
    while (curr->left != nullptr)  
        curr = curr->left;  
    return curr;  
}
```

```
Node* getSucc(Node* root, int target) {++++}
```

```
// Base Case 1: No Inorder Successor  
if (root == nullptr)  
    return nullptr;  
  
// Base Case 2: root is same as target and  
// right child is not empty  
if (root->data == target && root->right != nullptr)  
    return leftMost(root->right);  
  
// Use BST properties to search for  
// target and successor  
Node* succ = nullptr;  
Node* curr = root;  
while (curr != nullptr) {  
    // If curr node is greater, then it  
    // is a potential successor  
    if (target < curr->data) {  
        succ = curr;  
        curr = curr->left;  
    }  
  
    // If smaller, then successor must  
    // be in the right child  
    else if (target >= curr->data)  
        curr = curr->right;  
  
    // If same, the last visited  
    // greater value is the successor  
    else  
        break;  
}  
  
return succ;
```

BST or not

// Function to find max value in the subtree

```
int maxValue(Node* node) {  
    if (node == nullptr)  
        return INT_MIN;  
    return max({ node->data,  
                maxValue(node->left),  
                maxValue(node->right) });  
}
```

```
int minValue(Node* node) {  
    if (node == nullptr)  
        return INT_MAX;  
    return min({ node->data,  
                minValue(node->left),  
                minValue(node->right) });  
}
```

```
bool isBST(Node* node) {  
    if (node == nullptr)  
        return true;  
  
    if (node->left != nullptr && maxValue(node->left) >= node->data)  
        return false;  
  
    // Check if the min of the right subtree  
    // is smaller than or equal to current node  
    if (node->right != nullptr && minValue(node->right) <= node->data)  
        return false;  
  
    // Recursively check if left and right subtrees are BSTs  
    return isBST(node->left) && isBST(node->right);  
}
```

Node with Kth largest element

```
// A function to find k'th largest element in a given tree.
void kthLargestUtil(Node* root, int k, int& c)
{
    // Base cases, the second condition is important to
    // avoid unnecessary recursive calls
    if (root == NULL || c >= k)
        return;

    // Follow reverse inorder traversal so that the
    // largest element is visited first
    kthLargestUtil(root->right, k, c);

    // Increment count of visited nodes
    c++;

    // If c becomes k now, then this is the k'th largest
    if (c == k)
    {
        cout << "K'th largest element is "
              << root->key << endl;
        return;
    }

    // Recur for left subtree
    kthLargestUtil(root->left, k, c);
}

// Function to find k'th largest element
void kthLargest(Node* root, int k)
{
    // Initialize count of nodes visited as 0
    int c = 0;

    // Note that c is passed by reference
    kthLargestUtil(root, k, c);
}

/* A utility function to insert a new node with given key in BST */
Node* insert(Node* node, int key)
{
    /* If the tree is empty, return a new node */
    if (node == NULL) return newNode(key);

    /* Otherwise, recur down the tree */
    if (key < node->key)
        node->left = insert(node->left, key);
    else if (key > node->key)
        node->right = insert(node->right, key);

    /* return the (unchanged) node pointer */
    return node;
}
```

Least common ancestor

```
// Function to find LCA of n1 and n2 assuming
// both are present in the BST
Node* lca(Node* root, int n1, int n2) {
    if (root == nullptr)
        return nullptr;

    // If both n1 and n2 are smaller than
    // root, go to left subtree
    if (root->data > n1 && root->data > n2)
        return lca(root->left, n1, n2);

    // If both n1 and n2 are greater than
    // root, go to right subtree
    if (root->data < n1 && root->data < n2)
        return lca(root->right, n1, n2);

    // If one n1 or n2 is on one side
    // and the other on the other side,
    // root is the LCA
    return root;
}
```

Median

```
struct Node* newNode(int item)
{
    struct Node* temp = new Node;
    temp->data = item;
    temp->left = temp->right = NULL;
    return temp;
}

/* A utility function to insert a new node with
given key in BST */
struct Node* insert(struct Node* node, int key)
{
    /* If the tree is empty, return a new node */
    if (node == NULL) return newNode(key);

    /* Otherwise, recur down the tree */
    if (key < node->data)
        node->left = insert(node->left, key);
    else if (key > node->data)
        node->right = insert(node->right, key);

    /* return the (unchanged) node pointer */
    return node;
}
```

```

int counNodes(struct Node* root)
{
    struct Node* current, * pre;

    // Initialise count of nodes as 0
    int count = 0;

    if (root == NULL)
        return count;

    current = root;
    while (current != NULL)
    {
        if (current->left == NULL)
        {
            // Count node if its left is NULL
            count++;

            // Move to its right
            current = current->right;
        }
        else
        {
            /* Find the inorder predecessor of current */
            pre = current->left;

            while (pre->right != NULL &&
                pre->right != current)
                pre = pre->right;

            /* Make current as right child of its
            inorder predecessor */
            if (pre->right == NULL)
            {
                pre->right = current;
                current = current->left;
            }

            /* Revert the changes made in if part to
            restore the original tree i.e., fix
            the right child of predecessor */
            else
            {
                pre->right = NULL;

                // Increment count if the current
                // node is to be visited
                count++;
                current = current->right;
            } /* End of if condition pre->right == NULL */
        } /* End of if condition current->left == NULL */
    } /* End of while */

    return count;
}

```

```

using Morris Inorder Traversal*/
int findMedian(struct Node* root)
{
    if (root == NULL)
        return 0;

    int count = counNodes(root);
    int currCount = 0;
    struct Node* current = root, * pre, * prev;

    while (current != NULL)
    {
        if (current->left == NULL)
        {
            // count current node
            currCount++;

            // check if current node is the median
            // Odd case
            if (count % 2 != 0 && currCount == (count + 1) / 2)
                return current->data;

            // Even case
            else if (count % 2 == 0 && currCount == (count / 2) + 1)
                return (prev->data + current->data) / 2;

            // Update prev for even no. of nodes
            prev = current;

            // Move to the right
            current = current->right;
        }
        else
        {
            /* Find the inorder predecessor of current */
            pre = current->left;
            while (pre->right != NULL && pre->right != current)
                pre = pre->right;

            /* Make current as right child of its inorder predecessor */
            if (pre->right == NULL)
            {
                pre->right = current;
                current = current->left;
            }

            /* Revert the changes made in if part to restore the original
            tree i.e., fix the right child of predecessor */

```

```

/* Revert the changes made in if part to restore the original
tree i.e., fix the right child of predecessor */
else
{
    pre->right = NULL;

    prev = pre;

    // Count current node
    currCount++;

    // Check if the current node is the median
    if (count % 2 != 0 && currCount == (count + 1) / 2)
        return current->data;

    else if (count % 2 == 0 && currCount == (count / 2) + 1)
        return (prev->data + current->data) / 2;

    // update prev node for the case of even
    // no. of nodes
    prev = current;
    current = current->right;
} /* End of if condition pre->right == NULL */
} /* End of if condition current->left == NULL */
} /* End of while */
}

```

Sum of two vals equal to a num

```
bool findVal(Node* root, int x) {  
    if (root == nullptr) return false;  
  
    if (root->data == x) return true;  
    else if (root->data < x)  
        return findVal(root->right, x);  
    else  
        return findVal(root->left, x);  
}  
  
int countPairs(Node* root1, Node* root2, int x) {  
    if (root1 == nullptr) return 0;  
  
    int ans = 0;  
  
    // If pair (root1.data, x-root1.data) exists,  
    // then increment the ans.  
    if (findVal(root2, x - root1->data))  
        ans++;  
  
    // Recursively check for left and right subtree.  
    ans += countPairs(root1->left, root2, x);  
    ans += countPairs(root1->right, root2, x);  
  
    return ans;  
}
```

```

AvlTree Insert( ElementType X, AvlTree T ) {
    if ( T == NULL ) { /* Create and return a one-node tree */
        T = new AvlNode;
        T->Element = X;
        T->Left = T->Right = NULL;
    }
    else if( X < T->Element ) {
        T->Left = Insert( X, T->Left );
        if( Height( T->Left ) - Height( T->Right ) == 2 )
            if( X < T->Left->Element )
                T = SingleRotateWithLeft( T ); // RR rotation
            else
                T = DoubleRotateWithLeft( T ); // RL rotation
    }
    else if( X > T->Element ) {
        T->Right = Insert( X, T->Right );
        if( Height( T->Right ) - Height( T->Left ) == 2 )
            if( X > T->Right->Element )
                T = SingleRotateWithRight( T ); // LL rotation
            else
                T = DoubleRotateWithRight( T ); // LR rotation
    } /* Else X is in the tree already; we'll do nothing */
    T->Height = Max( Height( T->Left ), Height( T->Right ) ) + 1;
    return T;
}

```

20-AVL

Delete AVL Node

```
Node* deleteNode(Node* root, int key) {
    // Perform standard BST delete
    if (root == nullptr)
        return root;

    if (key < root->key)
        root->left = deleteNode(root->left, key);
    else if (key > root->key)
        root->right = deleteNode(root->right, key);
    else {
        // Node with only one child or no child
        if ((root->left == nullptr) || (root->right == nullptr)) {
            Node* temp = root->left ? root->left : root->right;
            if (temp == nullptr) {
                temp = root;
                root = nullptr;
            }
            else
                *root = *temp;
            delete temp;
        }
        else {
            // Node with two children
            Node* temp = minValueNode(root->right);
            root->key = temp->key;
            root->right = deleteNode(root->right, temp->key);
        }
    }
}
```

```

// If the tree has only one node, return it
if (root == nullptr)
    return root;

// Update height of the current node
root->height = 1 + max(height(root->left), height(root->right));

// Get balance factor to check if this node became unbalanced
int balance = getBalance(root);

// Balance the tree by rotations
// Left Left Case
if (balance > 1 && getBalance(root->left) >= 0)
    return rightRotate(root);

// Left Right Case
if (balance > 1 && getBalance(root->left) < 0) {
    root->left = leftRotate(root->left);
    return rightRotate(root);
}

// Right Right Case
if (balance < -1 && getBalance(root->right) <= 0)
    return leftRotate(root);

// Right Left Case
if (balance < -1 && getBalance(root->right) > 0) {
    root->right = rightRotate(root->right);
    return leftRotate(root);
}

return root;
}

```

Balance a BST

```
void storeInorder(Node* root, vector<int>& nodes) {
    if (root == nullptr) {
        return;
    }

    // Traverse the left subtree
    storeInorder(root->left, nodes);

    // Store the node data
    nodes.push_back(root->data);

    // Traverse the right subtree
    storeInorder(root->right, nodes);
}

// Function to build a balanced BST from a sorted array
Node* buildBalancedTree(vector<int>& nodes, int start, int end) {
    // Base case
    if (start > end) {
        return nullptr;
    }

    // Get the middle element and make it the root
    int mid = (start + end) / 2;
    Node* root = new Node(nodes[mid]);

    // Recursively build the left and right subtrees
    root->left = buildBalancedTree(nodes, start, mid - 1);
    root->right = buildBalancedTree(nodes, mid + 1, end);

    return root;
}
```

```
// Function to balance a BST
```

```
Node* balanceBST(Node* root) {
```

```
    vector<int> nodes;
```

```
    // Store the nodes in sorted order
```

```
    storeInorder(root, nodes);
```

```
    // Build the balanced tree from the sorted nodes
```

```
    return buildBalancedTree(nodes, 0, nodes.size() - 1);
```

```
}
```

```
// Function to print the tree (Inorder Traversal)
```

```
void inorder(Node* root) {
```

```
    if (root == nullptr) {
```

```
        return;
```

```
    }
```

```
    inorder(root->left);
```

```
    cout << root->data << " ";
```

```
    inorder(root->right);
```

```
}
```

```

void IntBinaryTree::insertNode(int num) {
    TreeNode *newNode, *nodePtr; // Pointer to create new node & traverse tree

    newNode = new TreeNode; // Create a new node
    newNode->value = num;
    newNode->left = newNode->right = NULL;

    if (!root) root = newNode; // If tree is empty.
    else { // Tree is not empty
        nodePtr = root;
        while (nodePtr != NULL) {
            if (num < nodePtr->value) { // Left subtree
                if (nodePtr->left) { nodePtr = nodePtr->left; }
                else { nodePtr->left = newNode; break; }
            }
            else if (num > nodePtr->value) { // Right subtree
                if (nodePtr->right) nodePtr = nodePtr->right;
                else { nodePtr->right = newNode; break; }
            }
            else { cout << "Duplicate value found in tree.\n"; break; }
        }
    }
}

```

```
void IntBinaryTree::deleteNode(int num, TreeNode *&nodePtr)
{
    if (nodePtr == NULL) // node does not exist in the tree
        cout << num << " not found.\n";
    else if (num < nodePtr->value)
        deleteNode(num, nodePtr->left); // find in left subtree
    else if (num > nodePtr->value)
        deleteNode(num, nodePtr->right); // find in right subtree
    else // num == nodePtr->value i.e. node is found
        makeDeletion(nodePtr); // actually deletes node from BST
}
```

```

void IntBinaryTree::makeDeletion(TreeNode *&nodePtr) {
    TreeNode *tempNodePtr; // Temporary pointer
    if (nodePtr->right == NULL) { // case for leaf and one (left) child
        tempNodePtr = nodePtr;
        nodePtr = nodePtr->left; // Reattach the left child
        delete tempNodePtr;
    }
    else if (nodePtr->left == NULL) { // case for one (right) child
        tempNodePtr = nodePtr;
        nodePtr = nodePtr->right; // Reattach the right child
        delete tempNodePtr;
    }
    else { // case for two children.
        tempNodePtr = nodePtr->right; // Move one node to the right
        while (tempNodePtr->left) { // Go to the extreme left node
            tempNodePtr = tempNodePtr->left;
        }
        tempNodePtr->left = nodePtr->left; // Reattach the left subtree
        tempNodePtr = nodePtr;
        nodePtr = nodePtr->right; // Reattach the right subtree
        delete tempNodePtr;
    }
}

```

Min difference with k

```
/* Utility that allocates a new node with the
   given key and NULL left and right pointers. */
struct Node* newNode(int key)
{
    struct Node* node = new (struct Node);
    node->key = key;
    node->left = node->right = NULL;
    return (node);
}

// Function to find node with minimum absolute
// difference with given K
// min_diff --> minimum difference till now
// min_diff_key --> node having minimum absolute
// difference with K
void maxDiffUtil(struct Node* ptr, int k, int& min_diff,
int& min_diff_key)
{
    if (ptr == NULL)
        return;

    // If k itself is present
    if (ptr->key == k)
    {
        min_diff_key = 0;
        return;
    }

    // update min_diff and min_diff_key by checking
    // current node value
    if (min_diff > abs(ptr->key - k))
    {
        min_diff = abs(ptr->key - k);
        min_diff_key = ptr->key;
    }

    // if k is less than ptr->key then move in
    // left subtree else in right subtree
    if (k < ptr->key)
        maxDiffUtil(ptr->left, k, min_diff, min_diff_key);
    else
        maxDiffUtil(ptr->right, k, min_diff, min_diff_key);
}

// Wrapper over maxDiffUtil()
int maxDiff(Node* root, int k)
{
    // Initialize minimum difference
    int min_diff = INT_MAX, min_diff_key = -1;

    // Find value of min_diff_key (Closest key
    // in tree with k)
    maxDiffUtil(root, k, min_diff, min_diff_key);

    return min_diff_key;
}
```

Common element in trees

```
// A utility function to create a new node
```

```
Node* newNode(int ele)
{
    Node* temp = new Node;
    temp->key = ele;
    temp->left = temp->right = NULL;
    return temp;
}
```

```
// A utility function to do inorder traversal
```

```
void inorder(struct Node* root, vector<int>& traversal)
{
    if (root) {
        inorder(root->left, traversal);
        traversal.push_back(root->key);
        inorder(root->right, traversal);
    }
}
```

```
// Function to print common elements in given two trees
```

```
void printCommon(Node* root1, Node* root2)
{
    vector<int> inorder1, inorder2;
    // Storing inorder traversal of both the trees
    inorder(root1, inorder1);
    inorder(root2, inorder2);

    cout << "Tree 1 : " << endl;
    for (int i = 0; i < inorder1.size(); i++) {
        cout << inorder1[i] << " ";
    }
    cout << endl;

    cout << "Tree 2 : " << endl;
    for (int i = 0; i < inorder2.size(); i++) {
        cout << inorder2[i] << " ";
    }
    cout << endl;
}
```

```

cout << "Common Nodes: " << endl;

// Using two pointers calculating common nodes in both the traversals
int i = 0, j = 0;
while (i < inorder1.size() && j < inorder2.size()) {
    if (inorder1[i] == inorder2[j]) {
        cout << inorder1[i] << " ";
        i++;
        j++;
    }
    else if (inorder1[i] < inorder2[j]) {
        i++;
    }
    else {
        j++;
    }
}

// A utility function to insert a new Node
// with given key in BST
struct Node* insert(struct Node* node, int key)
{
    // If the tree is empty, return a new Node
    if (node == NULL)
        return newNode(key);

    // Otherwise, recur down the tree
    if (key < node->key)
        node->left = insert(node->left, key);
    else if (key > node->key)
        node->right = insert(node->right, key);

    // Return the (unchanged) Node pointer
    return node;
}

```

```

Node* newNode(int data)
{
    Node* temp = new Node;
    temp->data = data;
    temp->left = temp->right = NULL;
    return temp;
}

/* A utility function to insert a new Node
with given key in BST */
struct Node* insert(struct Node* node, int key)
{
    /* If the tree is empty, return a new Node */
    if (node == NULL) return newNode(key);
    /* Otherwise, recur down the tree */
    if (key < node->data)
        node->left = insert(node->left, key);
    else if (key > node->data)
        node->right = insert(node->right, key);
    /* return the (unchanged) Node pointer */
    return node;
}

// Returns true if there is a dead end in tree,
// else false.
bool isDeadEndUtil(Node* root, int low, int high) {
    // Base case
    if (root == NULL)
        return false;
    // Check if current node falls within the range
    if (root->data >= low && root->data <= high)
        return true;
    // Recur for left and right subtrees
    return isDeadEndUtil(root->left, low, root->data - 1) || isDeadEndUtil(root->right, root->data + 1, high);
}

bool isDeadEnd(Node* root) {
    return isDeadEndUtil(root, 1, INT_MAX);
}

```

Finding a DeadEnd

Post from pre

```
void findPostOrderUtil(int pre[], int n, int minval,
    int maxval, int& preIndex)
{
    // If entire preorder array is traversed then
    // return as no more element is left to be
    // added to post order array.
    if (preIndex == n)
        return;

    // If array element does not lie in range specified,
    // then it is not part of current subtree.
    if (pre[preIndex] < minval || pre[preIndex] > maxval) {
        return;
    }

    // Store current value, to be printed later, after printing left and right subtrees. Increment
    // preIndex to find left and right subtrees, and pass this updated value to recursive calls.
    int val = pre[preIndex];
    preIndex++;

    // All elements with value between minval and val
    // lie in left subtree.
    findPostOrderUtil(pre, n, minval, val, preIndex);

    // All elements with value between val and maxval
    // lie in right subtree.
    findPostOrderUtil(pre, n, val, maxval, preIndex);

    cout << val << " ";
}

// Function to find postorder traversal.
void findPostOrder(int pre[], int n)
{
    // To store index of element to be traversed next in preorder array.
    // This is passed by reference to utility function.
    int preIndex = 0;

    findPostOrderUtil(pre, n, INT_MIN, INT_MAX, preIndex);
}
```

Output

BST is heap or not

```
// This function counts the
// number of nodes in a binary tree
int countNodes(Node* root) {
    if (root == nullptr)
        return 0;
    return 1 + countNodes(root->left) + countNodes(root->right);
}

// This function checks if the
// binary tree is complete or not
bool isCompleteUtil(Node* root, int index, int numberOfNodes) {
    if (root == nullptr)
        return true;

    // If index assigned to current node is more than
    // number of nodes in the tree,
    // then the tree is not complete
    if (index >= numberOfNodes)
        return false;

    // Recur for left and right subtrees
    return isCompleteUtil(root->left, 2 * index + 1, numberOfNodes) &&
        isCompleteUtil(root->right, 2 * index + 2, numberOfNodes);
}

// This function checks the heap property in the tree.
```

```

bool isHeapUtil(Node* root) {
    if (root->left == nullptr && root->right == nullptr)
        return true;

    // Node will be in the second-last level
    if (root->right == nullptr) {
        // Check heap property at the node
        // No recursive call because no need to
        // check the last level
        return root->key >= root->left->key;
    }
    else {
        // Check heap property at the node and recursively
        // check the heap property
        // at left and right subtrees
        if (root->key >= root->left->key && root->key >= root->right->key)
            return isHeapUtil(root->left) && isHeapUtil(root->right);
        else
            return false;
    }
}

// Function to check if the binary tree is
// a Heap or not.
bool isHeap(Node* root) {

    int nodeCount = countNodes(root);
    int index = 0;

    if (isCompleteUtil(root, index, nodeCount) && isHeapUtil(root))
        return true;
    return false;
}

```

Search key

```
Node* search(Node* root, int key) {  
  
    // Base Cases: root is null or key  
    // is present at root  
    if (root == NULL || root->key == key)  
        return root;  
  
    // Key is greater than root's key  
    if (root->key < key)  
        return search(root->right, key);  
  
    // Key is smaller than root's key  
    return search(root->left, key);  
}
```

```

int binarySearch(vector<int>& inorder, int l,
int r, int d) {
    int mid = (l + r) >> 1;

    if (inorder[mid] == d)
        return mid;

    else if (inorder[mid] > d)
        return binarySearch(inorder, l, mid - 1, d);

    else
        return binarySearch(inorder, mid + 1, r, d);
}

// Function to find Leaf Nodes by doing preorder
// traversal of tree using preorder and inorder vectors.
void leafNodesRec(vector<int>& preorder, vector<int>& inorder,
int l, int r, int& ind, vector<int>& leaves) {

    // If l == r, therefore no right or left subtree.
    // So, it must be leaf Node, store it.
    if (l == r) {
        leaves.push_back(inorder[l]);
        ind++;
        return;
    }

    // If array is out of bound, return.
    if (l < 0 || l > r || r >= inorder.size())
        return;

    // Finding the index of preorder element
    // in inorder vector using binary search.
    int loc = binarySearch(inorder, l, r, preorder[ind]);

    ind++;

    // Finding on the left subtree.
    leafNodesRec(preorder, inorder, l, loc - 1,
ind, leaves);

    // Finding on the right subtree.
    leafNodesRec(preorder, inorder, loc + 1, r,
ind, leaves);
}

```

Find leaf by pre via
post and in

```

// Finds leaf nodes from given preorder traversal.
vector<int> leafNodes(vector<int>& preorder) {
    int n = preorder.size();
    vector<int> inorder(n);
    vector<int> leaves;

    // Copy the preorder into another vector.
    for (int i = 0; i < n; i++)
        inorder[i] = preorder[i];

    // Finding the inorder by sorting the vector.
    sort(inorder.begin(), inorder.end());

    // Point to the index in preorder.
    int ind = 0;

    // Find the Leaf Nodes.
    leafNodesRec(preorder, inorder, 0, n - 1,
        ind, leaves);
    return leaves;
}

```

Path of max sum in BST

```
// Helper function to calculate the sum
// from a leaf node to the root
int getSumOfPath(Node* root, unordered_map<Node*, Node*>& parent) {

    // If the current node has no parent,
    // return its data (it's the root)
    if (parent.find(root) == parent.end())
        return root->data;

    // Recursively sum the current node data
    // with its parent's path sum
    return root->data + getSumOfPath(parent[root], parent);
}

// Function to calculate the maximum leaf-to-root path sum
void sumCal(Node* root, int& maxx, unordered_map<Node*, Node*>& parent) {

    // Check if the current node is a leaf node
    if (root->left == nullptr && root->right == nullptr) {

        // Update the maximum sum if the
        // current path's sum is greater
        maxx = max(maxx, getSumOfPath(root, parent));
        return;
    }

    // If the left child exists, set the
    // current node as its parent and recurse
    if (root->left) {
        parent[root->left] = root;
        sumCal(root->left, maxx, parent);
    }

    // If the right child exists, set the
    // current node as its parent and recurse
    if (root->right) {
        parent[root->right] = root;
        sumCal(root->right, maxx, parent);
    }
}
```

```
∨ // Function to return the maximum leaf-
  | // to-root path sum in the binary tree
∨ int maxPathSum(Node* root) {

    // To store each node's parent for path calculation
    unordered_map<Node*, Node*> parent;

    // Variable to store the maximum sum
    int maxx = 0;

    // Recursively calculate the sum for all paths
    sumCal(root, maxx, parent);

    // Return the maximum path sum found
    return maxx;
}
```

Find min

```
// in a vector
void inorder(Node* root, vector<int>& sortedInorder) {

    // Base Case
    if (root == nullptr) return;

    // Traverse Left subtree
    inorder(root->left, sortedInorder);

    // Store the current node's data
    sortedInorder.push_back(root->data);

    // Traverse right subtree
    inorder(root->right, sortedInorder);
}

int minValue(Node* root) {
    if (root == nullptr) {
        return -1;
    }

    vector<int> sortedInorder;

    // Call the recursive inorder function
    inorder(root, sortedInorder);

    // Return the first element, which is the minimum
    return sortedInorder[0];
}
```

number of unique BST with same dynamic key

```
// // Function to find number of unique BST
int numberOfBST(int n)
{
    // DP to store the number of unique BST with key i
    int dp[n + 1];
    fill_n(dp, n + 1, 0);

    // Base case
    dp[0] = 1;
    dp[1] = 1;

    // fill the dp table in bottom-up approach.
    for (int i = 2; i <= n; i++) {
        for (int j = 1; j <= i; j++) {
            // n-i in right * i-1 in left
            dp[i] = dp[i] + (dp[i - j] * dp[j - 1]);
        }
    }

    return dp[n];
}
```

if 2 elements give a certain sum in BST

```
✓ // Helper function to perform DFS and check
  // for the required sum.
✓ bool dfs(Node* root, unordered_set<int>& st, int sum) {
    if (root == nullptr)
        return false;

    // Check if the complement (sum - current node's value)
    // exists in the set
    if (st.count(sum - root->key))
        return true;

    // Insert the current node's value into the set
    st.insert(root->key);

    // Continue the search in the left and right subtrees
    return dfs(root->left, st, sum) ||
           dfs(root->right, st, sum);
}

✓ // Main function to check if two elements
  // in the BST sum to sum
✓ bool findTarget(Node* root, int sum) {
    // To store visited nodes' values
    unordered_set<int> st;

    // Start DFS from the root
    return dfs(root, st, sum);
}
```

Predecessor

```
void findPreSuc(Node* root, int key, Node*& pre,
                Node*& suc)
{
    // If root is null, return
    if (!root)
        return;

    // Traverse the Left subtree
    findPreSuc(root->left, key, pre, suc);

    // The first greater value seen is successor
    if (root->key > key)
    {
        if (!suc || (suc && suc->key > root->key))
            suc = root;
    }

    // The last smaller value seen is predecessor
    else if (root->key < key)
        pre = root;

    // Traverse the right subtree
    findPreSuc(root->right, key, pre, suc);
}
```

```
Node* insert(Node* node, int key)
{
    if (node == nullptr) return new Node(key);
    if (key < node->key)
        node->left = insert(node->left, key);
    else
        node->right = insert(node->right, key);
    return node;
}
```

if in a range (BST)

```
int getCount(Node *root, int l, int h) {  
  
    // Base case  
    if (root == nullptr) return 0;  
  
    // If current node is in range, then  
    // include it in count and recur for  
    // left and right children of it  
    if (root->data <= h && root->data >= l)  
        return 1 + getCount(root->left, l, h) +  
            getCount(root->right, l, h);  
  
    // If current node is smaller than low,  
    // then recur for right child  
    else if (root->data < l)  
        return getCount(root->right, l, h);  
  
    // Else recur for left child  
    else return getCount(root->left, l, h);  
}
```

remove-bst-keys-outside-the-given-range

```
// Function to remove keys that do not
// lie in the given range.
Node* removekeys(Node* root, int l, int r) {

    // base case
    if (root == nullptr) return nullptr;

    // Process the left subtree and get the
    // updated left node.
    Node* left = removekeys(root->left, l, r);

    // Process the right subtree and get the
    // updated right node.
    Node* right = removekeys(root->right, l, r);

    // If curr node lies in the range, update its
    // left and right nodes and return curr node.
    if (root->data >= l && root->data <= r) {
        root->left = left;
        root->right = right;

        return root;
    }

    // If current node is less than range,
    // then return the updated right subtree.
    else if (root->data < l) {
        return right;
    }

    // Else return the updated left subtree.
    else
        return left;
}

void printInorder(Node* root) {
    if (root == nullptr) return;

    printInorder(root->left);
    cout << root->data << " ";
    printInorder(root->right);
}
```

```
Node* newNode(int item)
```

```
{  
    Node* temp = new Node;  
    temp->data = item;  
    temp->left = temp->right = NULL;  
    return temp;  
}
```

replace-every-
element-with-the-
least-greater-
element-on-its-right

```
/* A utility function to insert a new node with  
given data in BST and find its successor */
```

```
Node* insert(Node* root, int val, int& suc)
```

```
{  
    /* If the tree is empty, return a new node */  
    if (!root)  
        return newNode(val);  
    // go to right subtree  
    if (val >= root->data)  
        root->right = insert(root->right, val, suc);  
    // If key is smaller than root's key, go to left  
    // subtree and set successor as current node  
    else {  
        suc = root->data;  
        root->left = insert(root->left, val, suc);  
    }  
    return root;  
}
```

```
// Function to replace every element with the least greater element on its right
```

```
void replace(int arr[], int n)
```

```
{  
    Node* root = nullptr;  
    // start from right to left  
    for (int i = n - 1; i >= 0; i--) {  
        int suc = -1;  
        // insert current element into BST and  
        // find its inorder successor  
        root = insert(root, arr[i], suc);  
        arr[i] = suc;  
    }  
}
```

Output

AVL Trees: LL Rotation

```
Position SingleRotateWithRight( Position K1 ) {  
    Position K2;  
    K2 = K1->Right; // K1: node whose balance factor is violated  
    K1->Right = K2->Left;  
    K2->Left = K1;  
    K1->Height = Max( Height(K1->Left), Height(K1->Right) ) + 1;  
    K2->Height = Max( Height(K2->Right), K1->Height ) + 1;  
    return K2; /* New root */  
}
```

K1 **K2**

AVL Trees: LR Rotation

Position **DoubleRotateWithRight**(Position K1)

```
{  
    /* RR rotation between K3 and K2 */  
    K1->Right = SingleRotateWithLeft( K1->Right );  
    /* LL rotation between K1 and K2 */  
    return SingleRotateWithRight( K1 );  
}
```

Single Right Rotation at K3

Single Left rotation at K1

AVL Trees: RL Rotation

```
Position DoubleRotateWithLeft( Position K3 )
{
    /* LL rotation between K1 and K2 */
    K3->Left = SingleRotateWithRight( K3->Left );
    /* RR rotation between K3 and K2 */
    return SingleRotateWithLeft( K3 );
}
```

Single Left Rotation at

AVL Trees: RR Rotation

```
Position SingleRotateWithLeft( Position K1 ) {  
    Position K2;  
    K2 = K1->Left; // K1: node whose balance factor is violated  
    K1->Left = K2->Right;  
    K2->Right = K1;  
    K1->Height = Max( Height(K1->Left), Height(K1->Right) ) + 1;  
    K2->Height = Max( Height(K2->Left), K1->Height ) + 1;  
    return K2; /* New root */  
}
```

Sorted array to BST

```
// Recursive function to construct BST
Node* sortedArrayToBSTRecur(vector<int>& arr,
    int start, int end) {
    if (start > end) return nullptr;

    // Find the middle element
    int mid = start + (end - start) / 2;

    // Create root node
    Node* root = new Node(arr[mid]);

    // Create left subtree
    root->left = sortedArrayToBSTRecur(arr, start,
        mid - 1);

    // Create right subtree
    root->right = sortedArrayToBSTRecur(arr, mid + 1,
        end);
    return root;
}

Node* sortedArrayToBST(vector<int>& arr) {
    int n = arr.size();
    return sortedArrayToBSTRecur(arr, 0, n - 1);
}

void preOrder(Node* root) {
    if (root == nullptr) return;
    cout << root->data << " ";
    preOrder(root->left);
    preOrder(root->right);
}
```

sum-of-descendant-nodes-below-target-in-binary-search-tree1

```
long long int res;  
// Variable to store the target node  
Node* target_node;
```

```
// Function to traverse down the BST and  
// calculate the sum based on the given direction  
void down(Node* node, int direction)
```

```
{  
    if (node) {  
        // Add the node's data to the result  
        // based on the direction  
        if (direction == 0)  
            res += node->data;  
  
        // Traverse left and right child  
        // nodes recursively  
        down(node->left, direction - 1);  
        down(node->right, direction + 1);  
    }  
}
```

```
// Function to find the target node and  
// calculate the sum vertically down from it  
void find(Node* node, int target)
```

```
{  
    // Check if the target node is null  
    if (target_node == NULL) {  
        if (node) {  
            // If the node's data is greater than  
            // the target, traverse left  
            if (node->data > target)  
                find(node->left, target);  
            // If the node's data is smaller  
            // than the target, traverse right  
            else if (node->data < target)  
                find(node->right, target);  
            // If the node's data is equal to  
            // the target, set target_node and  
            // calculate the sum vertically down  
            else {  
                target_node = node;  
                down(node, 0);  
            }  
        }  
    }  
}
```

```
// Function to calculate the sum vertically  
// down starting from the target node  
long long int verticallyDownBST(Node* root, int target)
```

```
{  
    // Initialize the result as -target and  
    // target_node as null  
    res = -target;  
    target_node = NULL;  
  
    // Find the target node and  
    // calculate the sum  
    find(root, target);  
  
    // If the sum is still -target, return -1  
    // as there is no valid target node  
    if (res == -target)  
        return -1;  
  
    // Return the final sum  
    return res;  
}
```